Macros + Homoiconicity

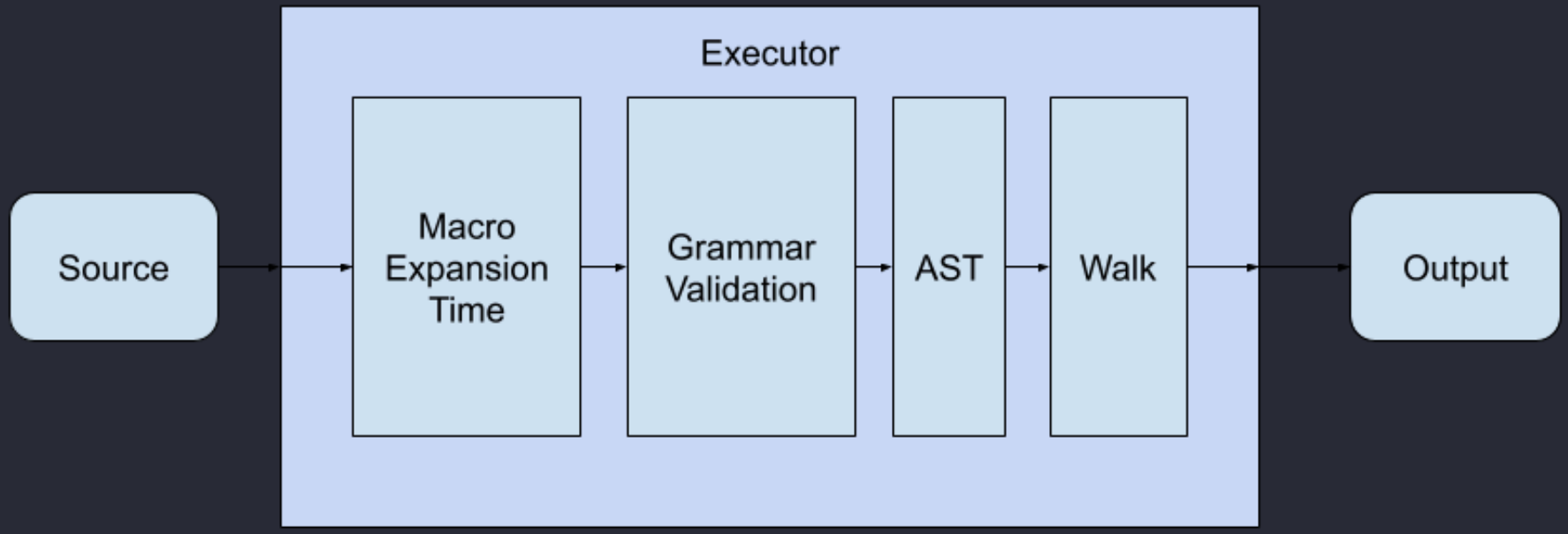
CS-3180

Macros

- Macros are just code that writes code
- Usually we think about code compiling into a lower level
- A type of *metaprogramming*
- Where a program can treat programs as data
- Supported in: Lisp, Rust, C/++, most ASM
- Unsupported in: Java, Python, JS, etc

Expansion

- Macros are code that *expand* into other code
- Expansions must be completed prior to execution
- Call this "Macro Expansion Time"



Types of Macros

- Two* main types of macros:
- *Textual Macros*
- Much simpler
- *Procedural Macros*
- Much more rich
- Take a look at both types today

Textual Macros

- Map a pattern to a predefined set of characters
- Text expansion via copy-paste
- Most commonly seen in C and C++
- Define a simple constants via `define`
- # for a *preprocessor directive*

```
#define NAME value
```

Textual Macros

- Every reference to `PI` gets `3.1415` copy-pasted into it
- (C devs call this an object-like macro 🙄)

```c
#include <stdio.h>
#define PI 3.14159
// pre-expansion main
int main() {
    float radius = 5.0;
    float area = PI * radius * radius;

    printf("Area of the circle: %.2f\n", area);
    return 0;
}
```

Textual Macros

- `PI` has been replaced
- Constant Propagation!

```c
#include <stdio.h>
#define PI 3.14159
// post-expansion main
int main() {
    float radius = 5.0;
    float area = 3.1415 * radius * radius;

    printf("Area of the circle: %.2f\n", area);
    return 0;
}
```

Textual Macro Arguments

- Textual macros can often take arguments
- This allowed them to act like inline functions
- But it is not actually a function
- Just a macro with a parameter
- Let's see that:

Function-Like Macro

- Still a copy-paste (no function calling)
- (C devs call these function-like macros)

```c
#include <stdio.h>

#define SQUARE(x) ((x) * (x))
//pre-expansion main
int main() {
    int num = 5;
    int squaredNum = SQUARE(num);

    printf("Square of 5: %d\n", squaredNum); // 25

    return 0;
}
```

Function-Like Macro

- Convention is to put everything in parentheses
- Help reduce bugs in replacement

```c
#include <stdio.h>

#define SQUARE(x) ((x) * (x))
//post-expansion main
int main() {
    int num = 5;
    int squaredNum = ((num) * (num));

    printf("Square of 5: %d\n", squaredNum); // 25

    return 0;
}
```

Limitations

- Find-Replace is a useful idea
- Adds a layer of complexity to programs
- Copy-Paste is a very limited process
- Let's really dive into what macros can do beyond this

Procedural Macros

- Proc Macros: Operate *directly on the AST*
- In Lisp, your code basically just is already an AST
- Features comes for free
- Enable much richer metaprogramming than simple replacement
- Terrible name ("Procedural" comes from "Procedures/Subroutines")

Quasi-Quoting

- To write richer macros, we need a way of separating:
- Code we want to take the value of
- Code we want to replace directly
- So we can properly mix them
- Enter the `quasiquote`

Quasi-Quoting

- The regular quote `'` expands to `quote`
- Allowed us to prevent evaluation, mostly to pass literals around
- The backtick `` ` `` expands `quasiquote`
- Works like `quote`, but data be *unquoted* later if needed

Quasi-Quoting

- The backtick `` ` `` un-evaluates
- The comma `,` re-evaluates
- Turns evaluation back on for whatever its in front of
- Return a literal list via `` ` ``
- But evaluate `user` (get its value)

```
(let ((user "Reese"))
  `("hello" ,user)) ; ("hello" "Reese")
```

Splicing

- Data that you want to evaluate
- is often given in list form
- You can "unpack" it from a list form using `,@`

```
(let ((value '(2 3)))
     `(1 ,value 4)) ; (1 (2 3) 4)

(let ((value '(2 3)))
     `(1 ,@value 4)) ; (1 2 3 4)
```

Lisp Macros

- With that separation, we can start writing macros
- `defmacro` function
- `(greet "Reese")` gets expanded before evaluation

```
1 (defmacro greet (name)
2   `(format t "Hello, ~a!~%" ',name))
3
4 (greet "Reese")
```

Lisp Macros

- We can use the `&body` construct
- Wraps all remaining arguments in a list
- `with-...` is a very common macro pattern

```
1  (defmacro with-logging (&body body)
2    `(progn
3       (format t "Starting code block...~%")
4       ,@body ; unpack body, so we don't execute as a list
5       (format t "Finished code block!~%")))
6
7  (with-logging
8      (let ((name "Reese"))
9          (format t "Hi ~S~%" name)))
10
```

Macro Expansion

- `macroexpand-1` function to see what a macro expands to
- (`macroexpand` was already taken by std. lib.)
- Useful for demo, not really for actual code
- Need a quote so it doesn't evaluate it

```
(format t "Macro expanded to ~S~%"
    (macroexpand-1 '(greet "Reese")))
```

Macros Simplification

- That's cool
- How would you do this with pure textual macros?
- Wrap your code in a lambda?
- Pass higher order functions, `funcall` later
- Macros allow for native support for things without needing to do that
- Let's see two more examples

My-When

- Many things in Lisp are secretly macros
- Including when, lets write it!

```
1  ; test is a specific arg
2  ; all other args are collected a (LIST ...) body
3  (defmacro my-when (test &body body)
4    `(if ,test
5         (progn
6           ,@body)))
7
8  (let ((input (read)))
9      (my-when (eq input 'TRUE)
10         (format t "You entered true")))
11
```

While

- Macros let you re-invent imperative paradigms
- In a functional way, lets write a `while`

```
(defmacro while (test &body body)
    `(if ,test
        (progn
            ,@body
            ; while expends body as list args
            ; so we still need an unpack
            (while ,test ,@body))
        nil)) ; when test fails, evaluate to nil ;-;

(let ((i 0))
    (while (< i 10)
        (format t "i was: ~S~%" i)
        (setf i (+ i 1))))
```

Homoiconicity

- Lisp can only get away this because code and data
- *share the same representation*
- Lisp Data = Cons Cells
- Lisp Code = Cons Cells in the AST
- This property is known as *homoiconicity*
- Mostly unique to Lisp
- Some languages can encode code as data via other methods

Lisp Executor

- Lisp specification doesn't state whether:
- Lisp should be compiled
- or Lisp should be interpreted
- Just that it needs a *macro-expansion time*
- Usually implemented via a compiler

Summary

- Macros are code that write code
- Simple textual macros are mostly find+replace
- Some can even take arguments
- Procedural macros are much more powerful
- Accomplish this by directly changing the AST

Questions?